Language models and building with them

What does a language model do, how do we test one, and how do we make it answer from our own documents?

AI and Data Science Professional | Session 22 of 24 | Module E: Neural networks and text
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Explain tokens, the context window, prompts and hallucination in plain words

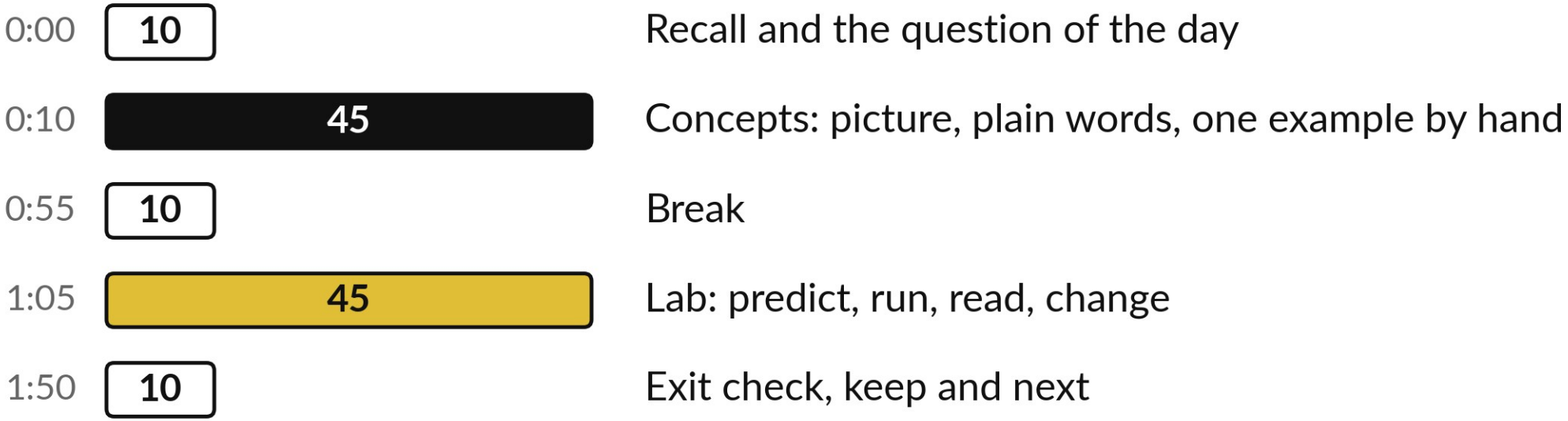


Build retrieval over a small document set and score it with precision at k and recall at k



Use AI tools safely: protect customer data and record AI-assisted work in an audit log

Your two hours: concepts first, then a Python lab on policy documents



Pictures and plain words first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. Why must text become numbers before a model can compare two messages?

2. What does TF-IDF do to a word that is found in almost every message?

3. What is semantic search meant to find that a keyword search misses?



Answer aloud first. Then we check together.

Answers to the three questions from Session 21

Question	Short answer
Why must text become numbers before a model can compare two messages?	A model only does arithmetic. A bag of words, TF-IDF or an embedding gives each message a vector it can measure.
What does TF-IDF do to a word that is found in almost every message?	It gives the word a weight near zero, so rare words count more.
What is semantic search meant to find that a keyword search misses?	Messages with a similar meaning but different words, such as crowded cashier and a long queue.

Words we use today

Word	What it means
Token and context window	A token is a word or piece of a word that a language model reads and writes. The context window is how many tokens it can read at once.
Prompt (system prompt, zero-shot, few-shot)	The instruction you give a model. A system prompt sets rules. Zero-shot gives no examples. Few-shot gives a few worked examples.
Hallucination	Fluent text that no source supports. It can read well and still be false.
Retrieval-augmented generation (RAG)	Find the relevant chunks of your documents first (retrieval and ranking), then ask the model to answer from them (grounding).
Evaluation set (precision at k, recall at k)	A labelled list of test questions with the right passage. Precision at k and recall at k score the search.

A language model reads and writes tokens: words, pieces of words and punctuation

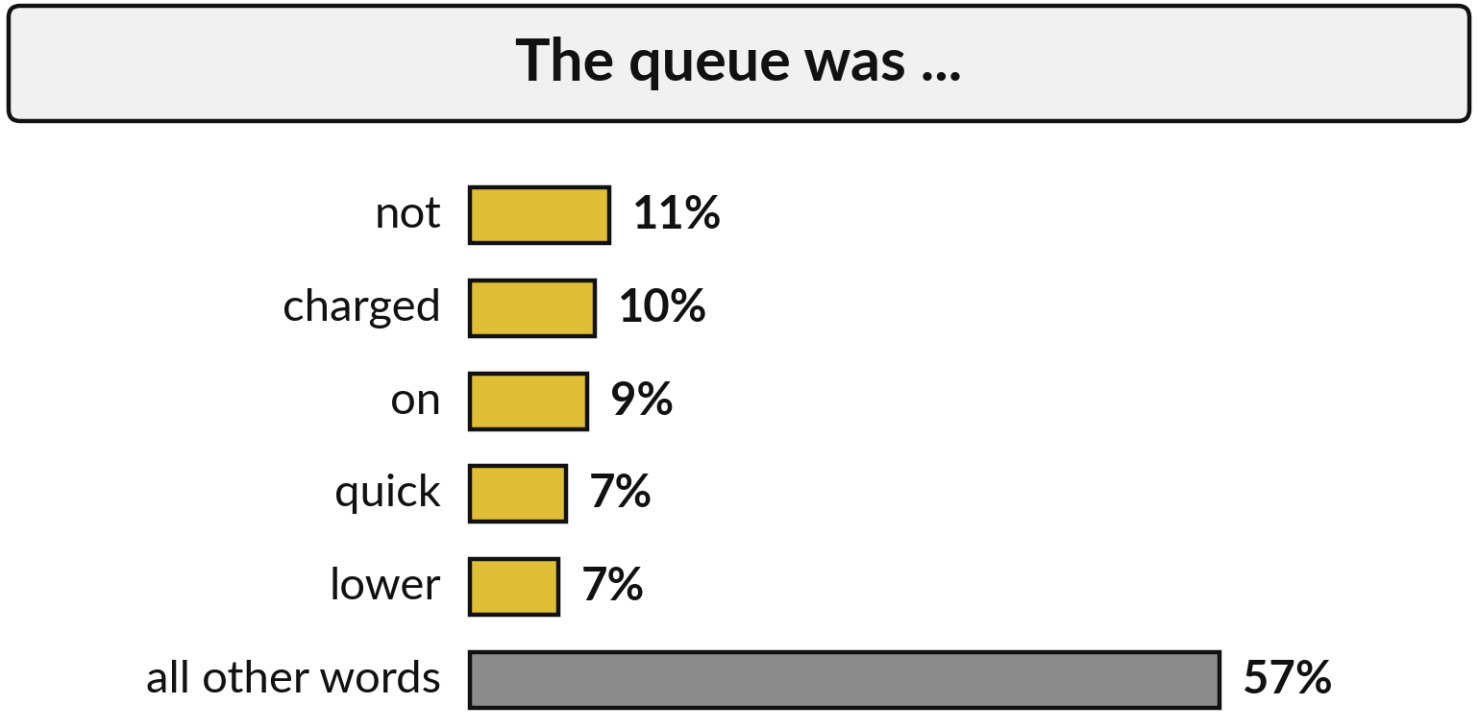
Text: Lakeside points expire after 12 months.



6 words became 8 tokens

Illustrative split and made-up numbers. A real tokeniser cuts and numbers text in its own way.

Next-token prediction: the model scores every possible next token and picks one



Counts of the word after "was" in all 179 cases in feedback.csv. The heading is only an example.

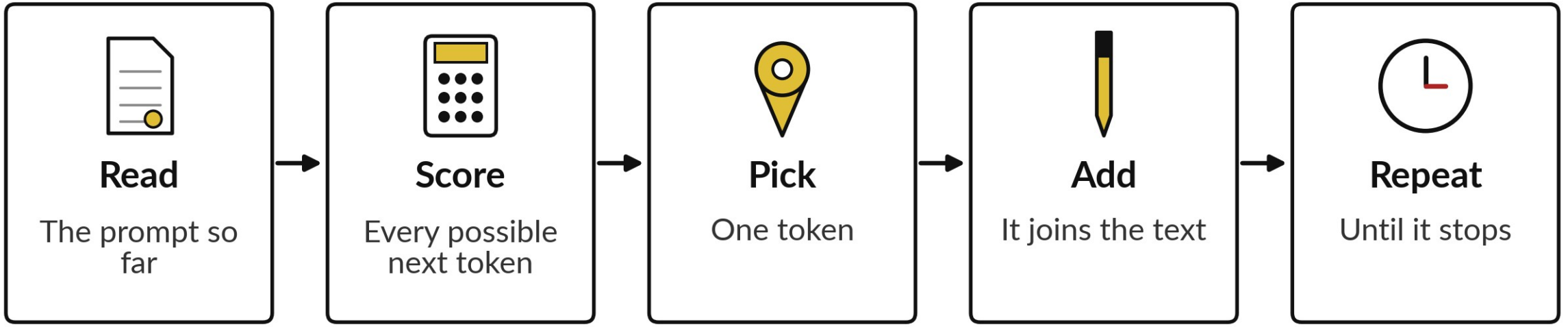
A language model is trained to predict the next token from the tokens before it.

The bars come from counting our own 800 messages.

A large model does the same job with a neural network that reads the whole context.

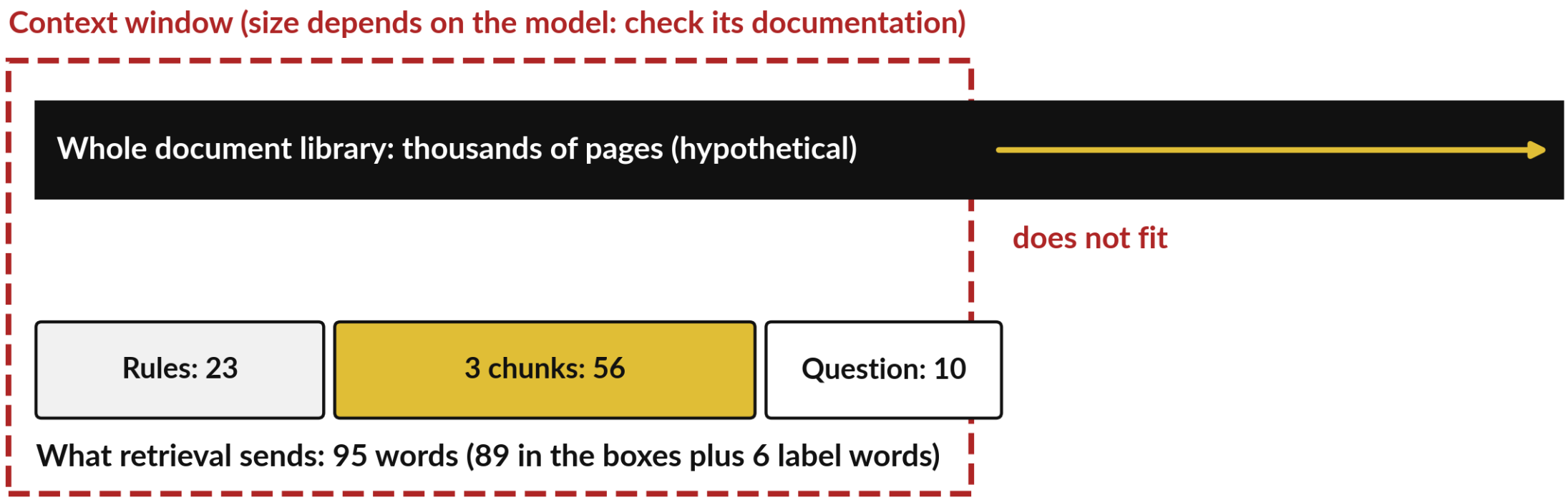
It scores every token it knows, then picks one.

One answer is built one token at a time: read, score, pick, add, repeat



A 40-token answer means 40 trips round this loop.

The context window limits how many tokens a model can read at once, so we send only what matters

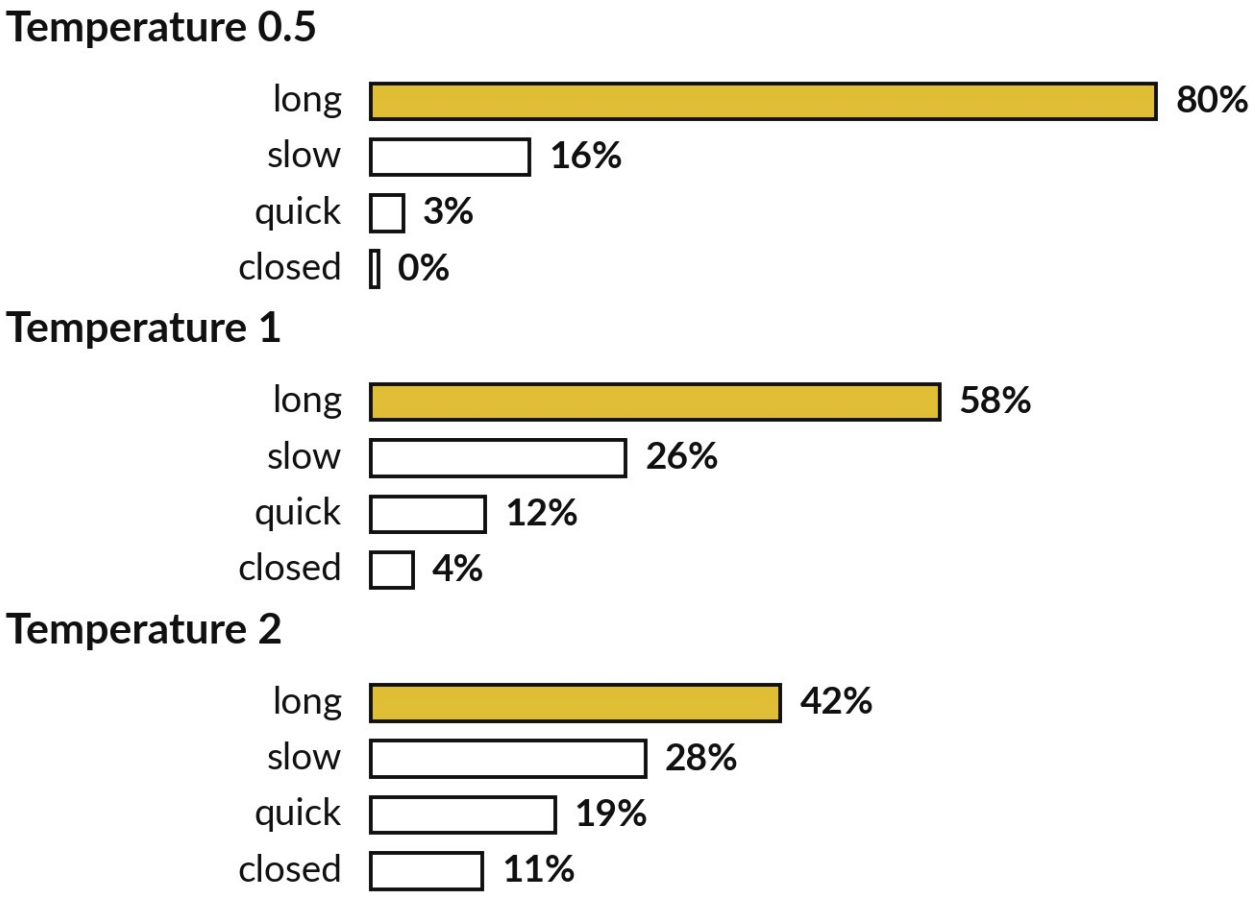


Prompt, pasted text and answer share one window. Word counts: our grounded prompt.

A prompt is your instruction: a system prompt sets rules, and examples turn zero-shot into few-shot

	Kind	What it is	Example from Lakeside Mart
	**System prompt**	Rules and role, set once for the whole conversation	Answer only from the passages. If the answer is missing, say so.
	**Zero-shot prompt**	A task with no worked examples	Label this message as positive, neutral or negative: Slow service at the counter.
	**Few-shot prompt**	A task with a few worked examples first	Fast checkout. Label: positive. Charged twice. Label: negative. Is milk available? Label:

Temperature is a setting that makes the next-token pick more repeatable or more varied



Many tools offer a setting called temperature. Check the tool documentation for what it does there.

Low temperature: the top token wins more often.

High temperature: the choice spreads out.

Illustrative made-up scores for The queue was …

Hallucination: a model can write a fluent answer that no source supports

No policy given

Question: How long do points last?

A model can still write a smooth answer with a guessed number of months (hypothetical).

Nothing can be checked.

Policy passage given

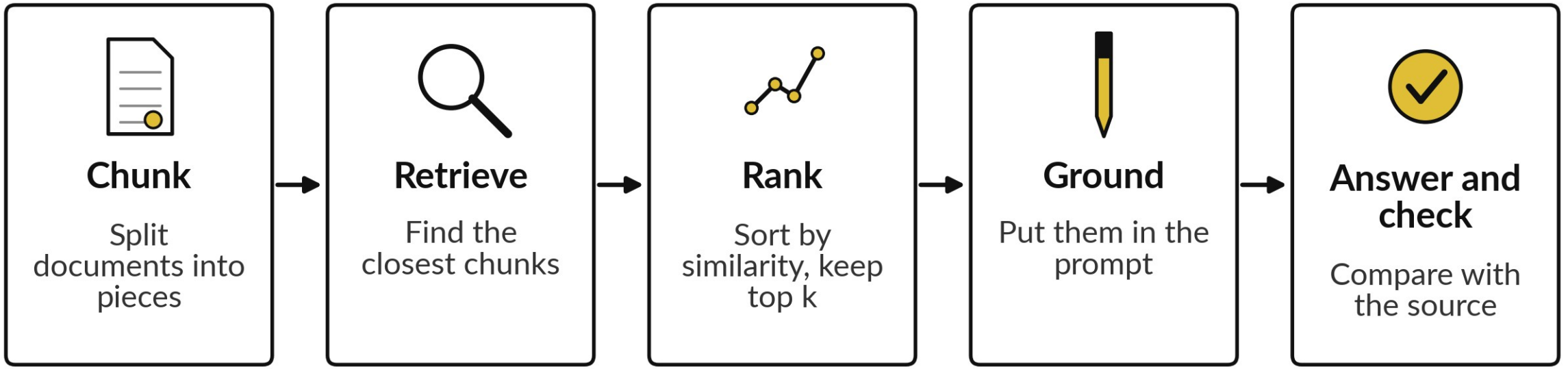
Passage: Points expire after 12 months.

The answer can quote the passage.

You can check it against the source.

Give the model the source. Check the answer.

Retrieval-augmented generation (RAG) finds the relevant chunks first, then answers from them



We build the first three steps in numpy today, and score them with an evaluation set.

Chunking splits each document into short pieces: the 8 policies become 59 chunks

loyalty_programme.txt

chunk 1	Lakeside Mart Loyalty Programme.
chunk 2	Every customer who gives a phone number at the till joins as Basic.
chunk 3	A customer earns 1 point for every UGX 1,000 spent.
chunk 4	Silver status starts at 2,000 points in 12 months and Gold at 6,000 points.
chunk 5	Silver customers receive 3 percent off selected household items on Saturdays.
chunk 6	Gold customers receive 5 percent off, free delivery within 5 kilometres and early notice of offers.
chunk 7	Points expire after 12 months.
chunk 8	Points are added within one day of purchase.
chunk 9	Points cannot be exchanged for cash.

59 chunks in all, about 12 words each (736 words)

Here one sentence is one chunk. Small chunks are precise. Large ones keep context.

Retrieval scores every chunk against the question, and ranking keeps the top k

	Rank	Chunk (file)	Cosine score
✓	1	Customers may return an item within 7 days of purchase with the till receipt. (returns_policy.txt)	0.56
✗	2	A customer who finds an expired item should return it with the receipt for a full refund and the branch records the incident. (food_safety.txt)	0.31
✗	3	Complaints from customers about a supplier's product are logged by item and branch and reviewed monthly by the buyer. (supplier_complaints.txt)	0.14

Question: "How many days do I have to return an item?" TF-IDF cosine similarity over 59 chunks and a vocabulary of 293 different words, as in Session 21.

Grounding: put the retrieved chunk in the prompt and tell the model to answer only from it

A grounded prompt with k = 1. The lab builds the same prompt with k = 3.

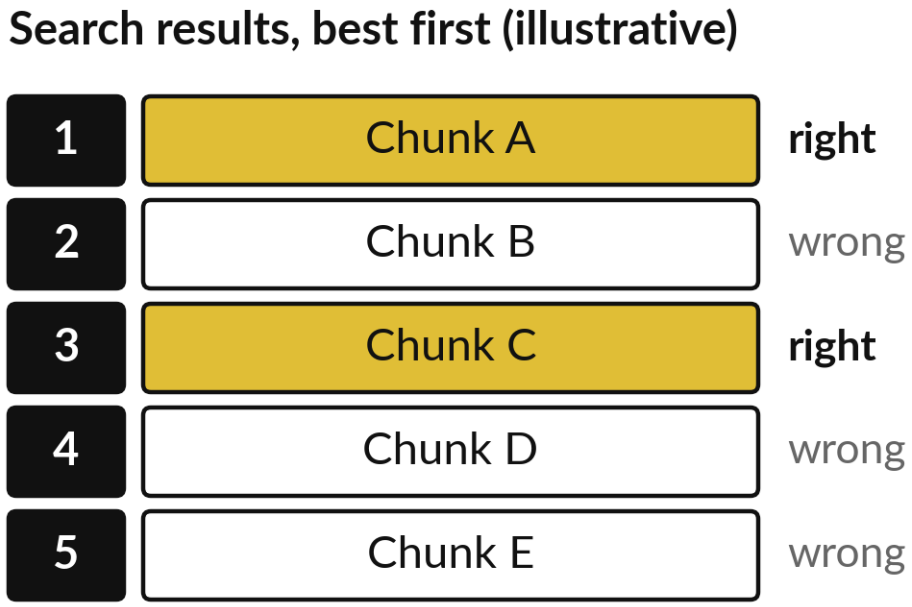
System prompt	Answer only from the passages below. If the answer is not in the passages, say: I could not find that in the policies.
Passage	Customers may return an item within 7 days of purchase with the till receipt.
Question	How many days do I have to return an item?

An evaluation set is a labelled list of test questions with the right passage for each

Question	Right policy file
How long do points last?	loyalty_programme.txt
How many days do I have to return an item?	returns_policy.txt
How long do mobile money refunds take?	returns_policy.txt
Which towns does delivery not reach?	delivery.txt
What is the delivery fee within 5 kilometres?	delivery.txt
What percent discount do Gold customers get?	loyalty_programme.txt

Question	Right policy file
How long is customer data kept?	data_privacy.txt
What temperature must dairy be stored at?	food_safety.txt
When do branches open on Sunday?	opening_hours.txt
What happens when more than 5 percent of a delivery is damaged?	supplier_complaints.txt
Does Lakeside Mart ask for my PIN?	mobile_money.txt
Can points be exchanged for cash?	loyalty_programme.txt

Precision at k and recall at k score the top k results of a search



Right chunks in the top 5: 2

Right chunks in the whole library: 3

Precision at k: how many of the top k results are right.
Recall at k: how many of all the right chunks were found.

Illustrative: 3 chunks are right in total.

Top 5 hold 2 of them: precision at 5 = 2/5 = 0.4.

Recall at 5 = 2/3 = 0.67.

Precision at k and recall at k are two ratios of the same count

Precision at k

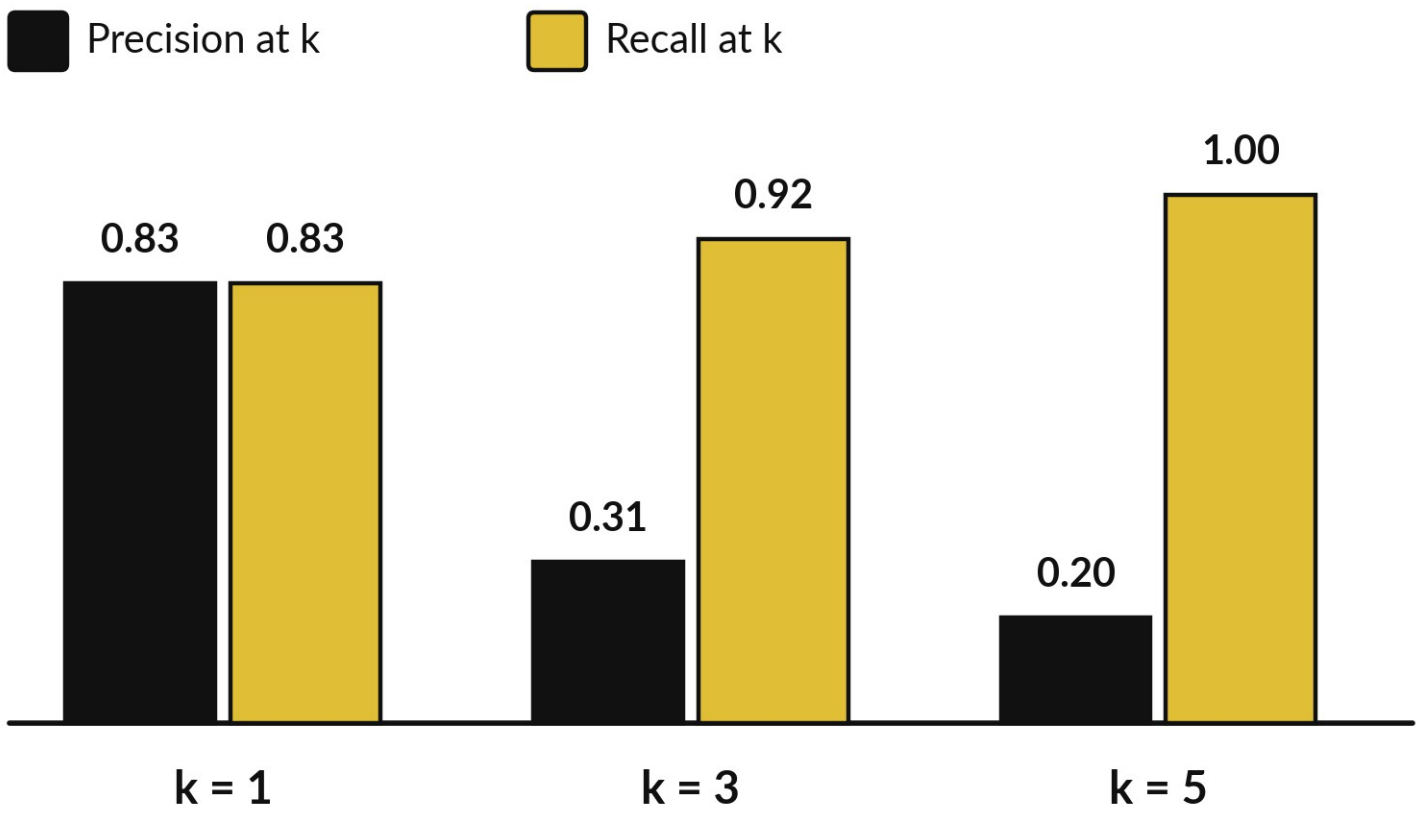
$$P@k = \frac{\text{relevant in top } k}{k}$$

Recall at k

$$R@k = \frac{\text{relevant in top } k}{\text{all relevant}}$$

Symbol	What it means	In our data
k	How many top results we look at	Here k = 3
Relevant in top k	Results in the top k that contain the answer	1 of 3 (return question)
All relevant	Chunks that contain the answer	1 per question here
Worked	Return question	P@3 = 1/3 = 0.33, R@3 = 1/1 = 1.00
Worked	Points question	right chunk at rank 4: P@3 = 0, R@3 = 0

On our twelve questions, keyword retrieval puts the right chunk first 10 times



Precision at 1 is 0.83.
Recall at 5 is 1.00.

Mean over 12 questions, TF-IDF cosine retrieval.

Precision falls as k grows, because only one chunk is right.

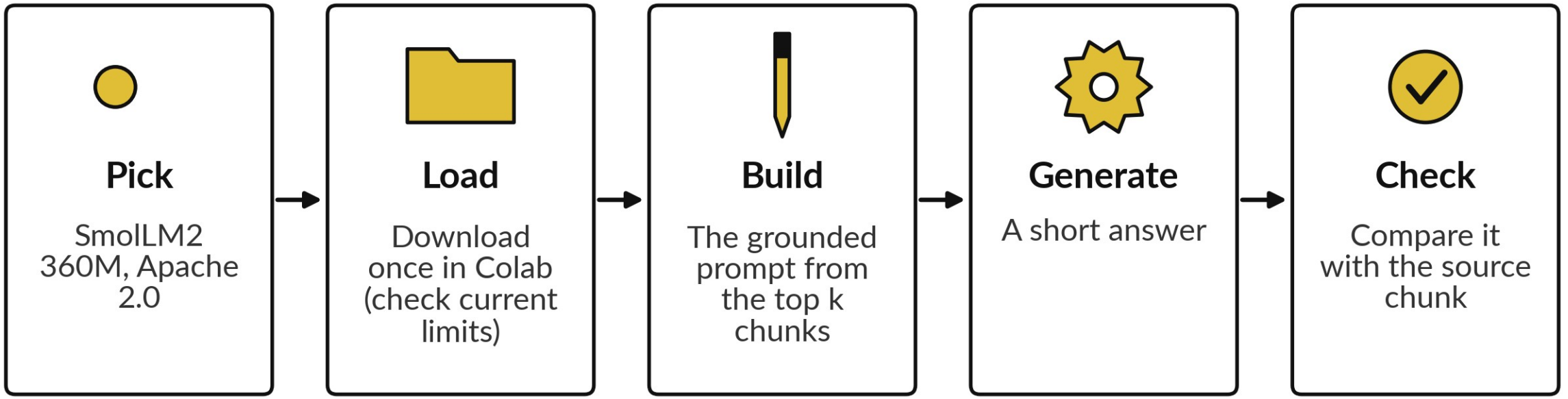
Recall rises as k grows.

Read the failures: two questions whose right chunk is not first, and why

	Question	Top result	Why it failed
✗	**How long do points last?**	Data is kept for 3 years after the last purchase. (data_privacy.txt)	The word last matches last purchase. The right chunk, Points expire after 12 months, says expire. It is at rank 4.
✗	**Which towns does delivery not reach?**	Lakeside Mart Delivery. (delivery.txt)	The title chunk repeats the word delivery. The right chunk is at rank 2.

Fixes to try: drop title chunks, add synonyms, or use embeddings. Then score again on the same set.

A small open model can run inside your own Colab notebook to write the grounded answer



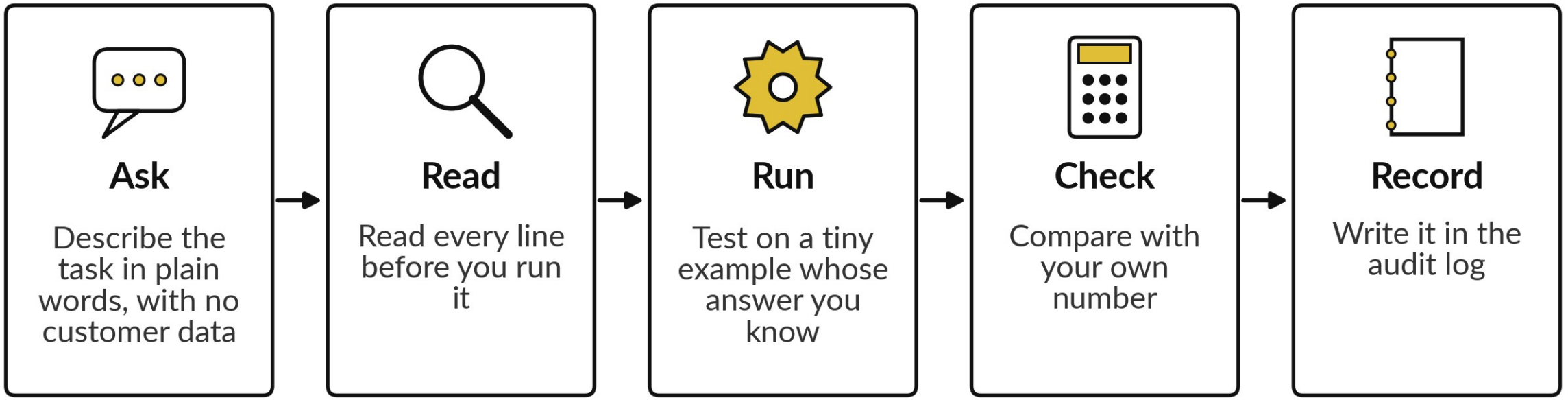
Optional lab step. Small models can be wrong or ignore rules, so checking is not optional.

Do not paste customer data into tools you do not control

What you want to paste	Into a tool you do not control?	What to do instead
Customer names, phone numbers, loan files	No	Remove or mask them first. Use customer numbers.
Passwords, keys and staff records	No	Keep them out of every prompt.
Public text, such as our eight policies	Usually fine	Check your organisation's rule first.
A made-up example with the same shape	Yes	Test the prompt on fake data, then run on real data inside your own system.

My understanding: under the Uganda Data Protection and Privacy Act, 2019 the data controller is responsible. Sending personal data abroad may need safeguards. Ask your data officer. Not legal advice.

AI-assisted coding is safe when you use it, check it and record it



An assistant can write code that runs and is still wrong. You are responsible for the result.

An audit log records what you asked, what you got and how you checked it

	What I asked	What it gave	How I checked	Decision
☐	**Split a policy text into sentences**	A regex split	Counted 59 chunks and matched two by hand	Used it
☐	**Explain recall at k**	A short explanation	Recomputed P@k and R@k in numpy	Kept the explanation
☐	**Group sales by branch in pandas**	Code with a wrong column name	It raised an error	Fixed it and noted it

Illustrative entries. Your lab saves audit_log.csv with the same columns.

Where you will use this at work: answer questions from your own documents, and test the answers

Your job	The documents you meet	What today lets you do
Compliance analyst at a bank, using Python	Internal policies and circulars	Find the right paragraph for a question and show its source, with a recall score.
Customer support lead at a retailer	FAQ and returns policy	Give agents the right chunk, and test the search on a list of real questions.
Records officer at a ministry	Regulations and circulars	Build a searchable index, then measure precision at k before you share it.
Programme officer at an NGO	Project reports	Keep an audit log of every AI-assisted summary and check each one against the report.

Read the recipe before the lab: four lines rank the chunks for one question

```
q = vec("how long do points last")
scores = T @ q
top = np.argsort(-scores, kind="stable")[:3]
for i in top:
    print(round(scores[i], 2), chunks[i][1][:30])
```

```
0.38 Data is kept for 3 years after
0.24 Points cannot be exchanged for
0.24 Silver status starts at 2,000
```

In plain words

First, vec, T and chunks come from lab steps 1 and 2. Line 1: the question becomes a TF-IDF vector.

Line 2: one cosine score per chunk.

Line 3: keep the three highest, the top k.

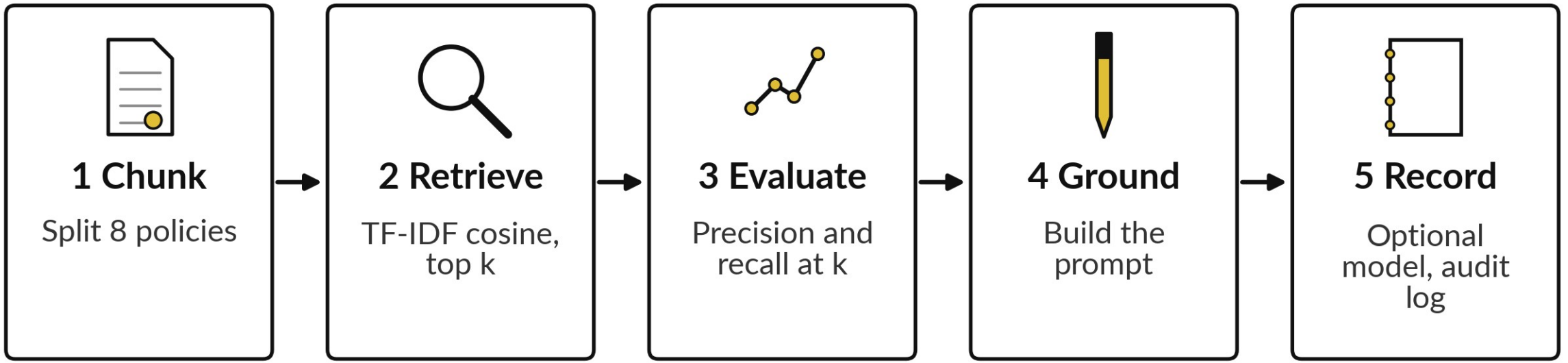
Lines 4 and 5: print score and chunk start.



Part 2: the Python lab

Close the slides. Open the notebook.

The lab has five parts, and you keep an evaluation file and an audit log



Five parts, ten tasks in 45 minutes. In each one: predict, run, read the output, change one thing.

Check your numbers before you move on

What you compute	You should get
Chunks and words in the eight policy files	59 and 736
Vocabulary size of the chunks (after stop words)	293
Cosine score of the top chunk for the return question	0.56, from returns_policy.txt
Mean precision at 1 and recall at 1 (12 questions)	0.833 and 0.833
Mean precision at 3 and recall at 3	0.306 and 0.917
Plain word overlap (count of shared question words), mean precision at 3	0.333
Rank of the right chunk for how long do points last	4
Words in the grounded prompt for the return question (k = 3)	95

If your number differs, find the step that went wrong. Do not change a number to match.

Five likely mistakes in the lab and how to fix them

	What you see	What it means	What to do
✗	**More than 59 chunks**	The splitter also cut after abbreviations or headings.	Split only after a full stop that is followed by a capital letter.
✗	**FileNotFoundError: policies**	The policy folder is not next to the notebook.	Run the first cell again and check that data.zip was unpacked.
✗	**Precision at 3 is 1.0**	You divided by the number of relevant chunks.	Precision divides by k. Recall divides by all relevant.
✗	**Every score is 0.0**	The question has no word in the vocabulary.	Check the stop words and the lowercase step.
✓	**The model ignores the passage**	A small model can ignore instructions.	Write it in your audit log, and trust the source chunk instead.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**The top 3 results hold 1 right chunk, and 4 chunks in the library are right. What are precision at 3 and recall at 3?**	Precision at 3 = 1/3 = 0.33. Recall at 3 = 1/4 = 0.25.
	**A model answers a policy question with a smooth sentence and no source. What is this called, and what do you do?**	Hallucination. Retrieve the passage, ground the prompt in it, and check the answer against the source.
	**May you paste a file of customer phone numbers into a free chat tool?**	No, unless your organisation has cleared the tool and a lawful purpose. It is personal data: mask or remove it first, and ask your data officer.

Save your notebook, your scores and your audit log

What you keep from today

Keep your notebook and, in the folder keep, eval_results.csv (your evaluation set with its scores) and audit_log.csv (what you asked an assistant, what it gave and how you checked it).

Our question today

What does a language model do, how do we test one, and how do we make it answer from our own documents?

Next session

Session 23: Ship it. We make work reproducible, save a model, build a small app and write a model card.

Take away

Your lab file

eval_results.csv

Your record

audit_log.csv

Next session

Ship it